Using CellTypist for cell type classification

This notebook showcases the cell type classification for scRNA-seq query data by retrieving the most likely cell type labels from either the built-in CellTypist models or the user-trained custom models.

Only the main steps and key parameters are introduced in this notebook. Refer to detailed Usage if you want to learn more.

Install CellTypist

```
!pip install celltypist
```

Show hidden output

```
import scanpy as sc
```

```
import celltypist
from celltypist import models
```

Download a scRNA-seq dataset of 2,000 immune cells

```
adata_2000 = sc.read('celltypist_demo_folder/demo_2000_cells.h5ad', backup_url = 'https://celltypist.cog.sanger.ac.uk/Notebook_demo_data/demo_2000_cells.h5ad')
```

This dataset includes 2,000 cells and 18,950 genes collected from different studies, thereby showing the practical applicability of CellTypist.

```
adata_2000.shape
```

The expression matrix (`adata_2000.X`) is pre-processed (and required) as log1p normalised expression to 10,000 counts per cell (this matrix can be alternatively stashed in `.raw.X`).

```
adata_2000.X.expm1().sum(axis = 1)
```

Some pre-assigned cell type labels are also in the data, which will be compared to the predicted labels from CellTypist later.

```
adata_2000.obs
```

Assign cell type labels using a CellTypist built-in model

In this section, we show the procedure of transferring cell type labels from built-in models to the query dataset.

Download the latest CellTypist models.

```
# Enabling `force_update = True` will overwrite existing (old) models.
models.download_models(force_update = True)
```

All models are stored in `models.models_path`.

```
models.models_path
```

Get an overview of the models and what they represent.

```
models.models_description()
```

Choose the model you want to employ, for example, the model with all tissues combined containing low-hierarchy (high-resolution) immune cell types/subtypes.

```
# Indeed, the `model` argument defaults to `Immune_All_Low.pkl`.
model = models.Model.load(model = 'Immune_All_Low.pkl')
```

Show the model meta information.

```
model
```

This model contains 98 cell states.

```
model.cell_types
```

Transfer cell type labels from this model to the query dataset using celltypist.annotate.

```
# Not run; predict cell identities using this loaded model.
#predictions = celltypist.annotate(adata_2000, model = model, majority_voting = True)
# Alternatively, just specify the model name (recommended as this ensures the model is intact every time it is loaded).
predictions = celltypist.annotate(adata_2000, model = 'Immune_All_Low.pkl', majority_voting = True)
```

By default (`majority_voting = False`), CellTypist will infer the identity of each query cell independently. This leads to raw predicted cell type labels, and usually finishes within seconds or minutes depending on the size of the query data. You can also turn on the majority-

voting classifier (`majority_voting = True`), which refines cell identities within local subclusters after an over-clustering approach at the cost of increased runtime.

The results include both predicted cell type labels (`predicted_labels`), over-clustering result (`over_clustering`), and predicted labels after majority voting in local subclusters (`majority_voting`). Note in the `predicted_labels`, each query cell gets its inferred label by choosing the most probable cell type among all possible cell types in the given model.

```
predictions.predicted_labels
```

Transform the prediction result into an `AnnData`.

```
# Get an `AnnData` with predicted labels embedded into the cell metadata columns.
adata = predictions.to_adata()
```

Compared to `adata_2000`, the new `adata` has additional prediction information in `adata.obs` (`predicted_labels`, `over_clustering`, `majority_voting` and `conf_score`). Of note, all these columns can be prefixed with a specific string by setting `prefix` in to_adata.

```
adata.obs
```

In addition to this meta information added, the neighborhood graph constructed during over-clustering is also stored in the `adata` (If a pre-calculated neighborhood graph is already present in the `AnnData`, this graph construction step will be skipped).
This graph can be used to derive the cell embeddings, such as the UMAP coordinates.

```
# If the UMAP or any cell embeddings are already available in the `AnnData`, skip this command.
sc.tl.umap(adata)
```

Visualise the prediction results.

```
sc.pl.umap(adata, color = ['cell_type', 'predicted_labels', 'majority_voting'], legend_loc = 'on data')
```

Actually, you may not need to explicitly convert `predictions` output by `celltypist.annotate` into an `AnnData` as above. A more useful way is to use the visualisation function celltypist.dotplot, which quantitatively compares the CellTypist prediction result (e.g. `majority_voting` here) with the cell types pre-defined in the `AnnData` (here `cell_type`). You can also change the value of `use_as_prediction` to `predicted_labels` to compare the raw prediction result with the pre-defined cell types.

```
celltypist.dotplot(predictions, use_as_reference = 'cell_type', use_as_prediction = 'majority_voting')
```

For each pre-defined cell type (each column from the dot plot), this plot shows how it can be 'decomposed' into different cell types predicted by CellTypist (rows).

Assign cell type labels using a custom model

In this section, we show the procedure of generating a custom model and transferring labels from the model to the query data.

Use previously downloaded dataset of 2,000 immune cells as the training set.

```
adata_2000 = sc.read('celltypist_demo_folder/demo_2000_cells.h5ad', backup_url = 'https://celltypist.cog.sanger.ac.uk/Notebook_demo_data/demo_2000_cells.h5ad')
```

Download another scRNA-seq dataset of 400 immune cells as a query.

```
adata_400 = sc.read('celltypist_demo_folder/demo_400_cells.h5ad', backup_url = 'https://celltypist.cog.sanger.ac.uk/Notebook_demo_data/demo_400_cells.h5ad')
```

Derive a custom model by training the data using the celltypist.train function.

```
# The `cell_type` in `adata_2000.obs` will be used as cell type labels for training.
new_model = celltypist.train(adata_2000, labels = 'cell_type', n_jobs = 10, feature_selection = True)
```

Refer to the function celltypist.train for what each parameter means, and to the usage for details of model training.

This custom model can be manipulated as with other CellTypist built-in models. First, save this model locally.

```
# Save the model.
new_model.write('celltypist_demo_folder/model_from_immune2000.pkl')
```

You can load this model by `models.Model.load`.

```
new_model = models.Model.load('celltypist_demo_folder/model_from_immune2000.pkl')
```

Next, we use this model to predict the query dataset of 400 immune cells.

```
# Not run; predict the identity of each input cell with the new model.
#predictions = celltypist.annotate(adata_400, model = new_model, majority_voting = True)
# Alternatively, just specify the model path (recommended as this ensures the model is intact every time it is loaded).
predictions = celltypist.annotate(adata_400, model = 'celltypist_demo_folder/model_from_immune2000.pkl', majority_voting = True)
```

```
adata = predictions.to_adata()
```

```
sc.tl.umap(adata)
```

```
sc.pl.umap(adata, color = ['cell_type', 'predicted_labels', 'majority_voting'], legend_loc = 'on data')
```

```
celltypist.dotplot(predictions, use_as_reference = 'cell_type', use_as_prediction = 'majority_voting')
```

Examine expression of cell type-driving genes

Each model can be examined in terms of the driving genes for each cell type. Note these genes are only dependent on the model, say, the training dataset.

```
# Any model can be inspected.
# Here we load the previously saved model trained from 2,000 immune cells.
model = models.Model.load(model = 'celltypist_demo_folder/model_from_immune2000.pkl')
```

```
model.cell_types
```

Extract the top three driving genes of `Mast cells` using the extract_top_markers method.

```
top_3_genes = model.extract_top_markers("Mast cells", 3)
top_3_genes
```

```
# Check expression of the three genes in the training set.
sc.pl.violin(adata_2000, top_3_genes, groupby = 'cell_type', rotation = 90)
```

```
# Check expression of the three genes in the query set.
# Here we use `majority_voting` from CellTypist as the cell type labels for this dataset.
sc.pl.violin(adata_400, top_3_genes, groupby = 'majority_voting', rotation = 90)
```